Contents

Enterprise Cloud and Web Applications 2
Introduction 2
Architecture Diagram 2
AWS services being used 3
Amazon S3 3
AWS Lambda 3
AWS DynamoDB 3
AWS SNS 4
Lambda Implementation 4
Dependency Packaging 4
Tested and Validation 4
Challenges Faced 5
My Critical Evaluation 5
Conclusion 5
Appendix 6
Code of Lambda Function code 6
Code used for IAM Policy 8
Screenshots 10

Figure 1: Architecture Diagram 2
Figure 2: S3 Bucket created 10
Figure 3: IAM Users created 11
Figure 4: DynamoDB table created 11
Figure 5: SNS Subscription created 11
Figure 6: SNS Subscription E-Mail Confirmation 12
Figure 7: Triggers for companyA, companyB, company 12
Figure 8: Lambda Function Created 13
Figure 9: Log Streams after each test and uploads of excel file 13
Figure 10: Created a code file in the folder 14
Figure 11: Installed the necessary python dependencies 14
Figure 12: Test SNS E-Mail 15

Enterprise Cloud and Web Applications

Introduction

The growth of serverless architecture has made it possible for apps made for today's businesses to have a comparatively more affordable, scalable, and maintenance-free set of services as the cloud computing paradigm shifts. In this case, our project chose to use AWS to create a serverless workflow in order to meet the module assignment brief's Scenario 1 requirements.

Businesses were required to submit Excel workbooks to an Amazon S3 bucket. The content of the file would be extracted and stored in a structured database by a process that would be started upon upload. Additionally, the system was supposed to notify the stakeholders via email that the procedure was successful. Using S3, Lambda, DynamoDB, and SNS, this was put into practice. The method was written in Python and parsed Excel spreadsheets using openpyxl.

Architecture design, services used, process of implementation, problems encountered, testing, and a critical assessment of the solution are all covered in this special presentation.

Architecture Diagram

All levels of serverless models are guaranteed by the architecture, allowing for scalability, reduced operational responsibilities, and extremely low costs.

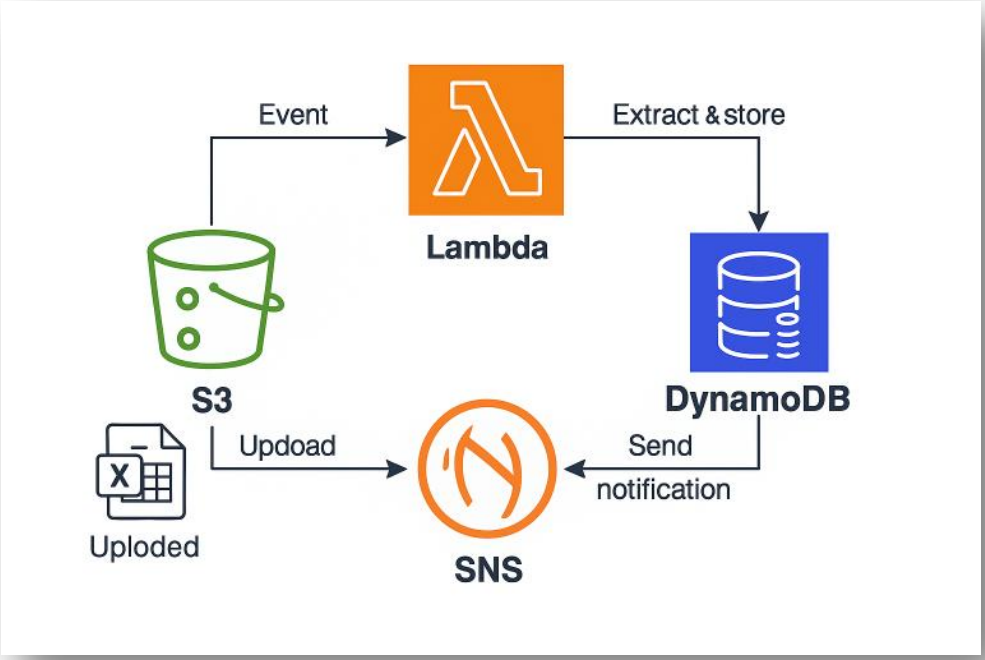

Figure 1: Architecture Diagram

The below explains the Architecture above:

1. Amazon S3 stores the uploaded Excel Files
2. AWS Lambda is triggered automatically by an S3 “ObjectCreated” event
3. Lambda
    a. Downloads the Excel file using the boto3.
    b. Uses openpyxl to extract the rows from the Excel file.
    c. Inserts data into Amazon DynamoDB
    d. Sends a notification via Amazon SNS
4. DynamoDB acts as the central NoSQL database to store extracted data
5. SNS notifies the subscribed users via E-Mail.

AWS services being used

Amazon S3

The primary source for Excel file submissions from outside users is the Amazon S3 bucket "xyz-logistics-uploads-chamith." Every time a fresh file gets uploaded, it automatically calls AWS Lambda functions using S3 event alerts. Logical prefixes such as companyA/, companyB/ and company/C are used to organise the bucket. This helps to ensure precise file identification and handling for each organisation by facilitating the separation of organisational data uploads.

AWS Lambda

The foundation of the serverless processing system is the AWS Lambda function. Python 3.12 was utilised in its creation. Because no containers or server needs to be handled, it sets up the work necessary for automatically executing code in response to events in Amazon S3. Important elements like the DynamoDB table name (DYNAMO_TABLE) and SNS topic ARN (SNS_TOPIC_ARN) were kept as environment variables to maintain its flexibility and configurability.

This implies that no modifications to the code are needed to support any of the environments in which the function runs. Additionally, the openpyxl library, which can be found on external libraries, was needed in order to read Excel files using the lambda function. Pip install -t was used to install the library locally because it is not by default available on the Lambda runtime. To ensure effective deployment and execution within Lambda, it was also crucial to thoroughly zip the function code with all dependencies at the root level of the ZIP archive.

AWS DynamoDB

The NoSQL database design of Amazon DynamoDB made it possible to store user-uploaded modified Excel files. To identify each company-related data row from each unique upload session, the LogisticsUploadRecords database uses CompanyID as the partition key and RowID as the sort key. Without requiring a predetermined schema, the table saves Excel file information, including column headers, automatically through dynamic mapping operations. DynamoDB's adaptability made it the ideal choice for managing Excel structures used by many businesses. The serverless application that relies on events was ideally matched with the system's features, which include high scalability, rapid read and write operations, and outstanding availability.

AWS SNS

Following successful Excel file processing, instant email alerts were delivered via Amazon Simple Notification Service (SNS). These notifications were handled by the system using a topic named NewUploadNotification. This topic was used to send a message containing the company name and the extracted row count once the AWS Lambda function had completed processing and saving the Excel data. Multiple email addresses were subscribed to the SNS subject in order to provide crucial stakeholders with instant notifications. By implementing this integration, the need for manual monitoring was removed while preserving a high degree of user awareness and system responsiveness.

Lambda Implementation

The AWS Lambda function was considered crucial by the serverless system for managing S3 file upload events. The system downloaded Excel files using Python 3.12, parsed them using openpyxl, saved the contents to DynamoDB, and utilised SNS to generate summary notifications. The system's versatility was preserved by handling configuration values through environment variables. This method offers the system readily scalable and maintainable automated data processing capabilities. The appendix contains the code for this system.

The codes being used has being added in the appendix section in this report.

FrontEnd UI

In order to develop the FrontEnd UI, I first generated a new S3 bucket, then I used Visual Studio Code to create an index.html file and an upload.js file. I then uploaded these two files into the newly created S3 bucket, and the front end was operational.

Dependency Packaging

Before the AWS Lambda function could process Excel files, the openpyxl library had to be added as a third-party package. Before packaging the function code, openpyxl must be manually installed because it is not included in the default AWS Lambda runtime. In order to install openpyxl and its dependencies in a local folder, the pip command and the -t flag had to be used to direct installation to that location. After that, all files had to be compressed at the root level (without any subfolders).

Using the terminal, the compressed ZIP file was manually moved to AWS Lambda. To guarantee correct execution, the ZIP structure has to contain both the openpyxl package and lambda_function.py at the primary level. One important precaution that stopped ImportModuleError from happening was the packaging process. This document's appendix contains the full packing instructions.

The code used in the terminal are taken as screenshots and added in the appendix section.

Tested and Validation

In order to verify the system's predicted performance, a dual test approach that coupled automated processes with manual evaluations was necessary. The S3 bucket name and object key for validation were included in a test event created using the AWS Lambda console during manual testing. As CloudWatch logs verified these processing stages, the system processed files using Lambda's execution, which caused the S3 bucket to deliver files to DynamoDB for insertion and to send SNS email alerts.

The group used the AWS console to manually upload.xlsx files to the S3 bucket in order to carry out automated trigger testing. The team used this procedure to confirm that the Lambda function launched automatically and processed data accurately, and that the recipient received the notification email a few seconds after the file was uploaded. The primary monitoring and debugging tools, such as CloudWatch, were used to identify and address problems such dependency packaging errors, object key mismatches, and InvalidFileException. The full application of testing techniques gave the serverless process resilience and dependability.

Challenges Faced

Challenge	Solution
Lambda couldn't find openpyxl	Corrected packaging structure-ensured dependencies were zipped at root.
InvalidFileException from openpyxl	Verified .xlsx format and reuploaded valid Excel files
Wrong object key in test event	Used exact S3 key including prefix and suffix and spacing
S3 trigger not working	Fixed event prefix (company/A) and suffix (.xlsx)
SNS not sending email	Confirmed correct topic ARN E-mail and Subscription

Table 1: Challenges Faced

My Critical Evaluation

Through this project, I was able to explore utilising AWS resources while putting serverless architecture into effect in practical settings. The main advantage of the system was its fully serverless architecture, which facilitated autonomous scalability and rendered infrastructure administration obsolete. Because it only used resources when necessary, the pay-per-use model kept the solution affordable. High degrees of component independence were incorporated into the system's architecture to provide simple maintenance and operational adaptability. During its execution, the project ran into certain limitations. Openpyxl dependencies in the system needed specific management, which made Lambda packaging activities difficult.

I had the chance to experiment using AWS technologies while putting serverless architecture into effect in real-world scenarios thanks to this project. Its full serverless architecture, which permitted autonomous scaling and rendered infrastructure administration obsolete, was the system's main advantage. Because the pay-per-use model only used resources when necessary, the solution remained affordable. In order to provide easy maintenance and operational flexibility, the system's architecture established high levels of independence across its components. The project ran into certain limitations as it was being implemented. Lambda packing activities were complicated by the system's need for particular handling of openpyxl dependencies.

Conclusion

The system has fully complied with all of the functional criteria of Scenario 1. Stakeholders were notified via email when the AWS serverless solution processed automatic Excel file uploads for content extraction through DynamoDB storage. I learnt more about event-driven, cloud-native methodologies

from this work. I developed AWS service dependencies and learnt how to manage IAM and deploy Lambda functions with CloudWatch debugging techniques.

The serverless computing project offers a real-world application example that enables developers to construct contemporary enterprise cloud solutions with less code and without the need to manage servers.

Appendix

Code of Lambda Function code

```
import boto3

import os

import json

from openpyxl import load_workbook

from datetime import datetime

import tempfile


# AWS clients

s3 = boto3.client('s3')

dynamodb = boto3.resource('dynamodb')

sns = boto3.client('sns')


# Environment variables (set these later in Lambda config)

DYNAMO_TABLE = os.environ['DYNAMO_TABLE']

SNS_TOPIC_ARN = os.environ['SNS_TOPIC_ARN']


def lambda_handler(event, context):

    for record in event['Records']:

        bucket_name = record['s3']['bucket']['name']

        file_key = record['s3']['object']['key']

        company_id = file_key.split('/')[0]  # e.g., 'companyA'


        # Download file from S3 to temp file

        tmp_file = tempfile.mktemp()

        s3.download_file(bucket_name, file_key, tmp_file)
```

```
# Load Excel
workbook = load_workbook(tmp_file)
sheet = workbook.active
rows = list(sheet.iter_rows(values_only=True))
headers = rows[0]
data_rows = rows[1:]

# Connect to DynamoDB
table = dynamodb.Table(DYNAMO_TABLE)
timestamp = datetime.utcnow().isoformat()

# Loop through rows and insert
for i, row in enumerate(data_rows):
    item = {
        'CompanyID': company_id,
        'UploadTimestamp': timestamp,
        'RowID': str(i)
    }
    for col_index, value in enumerate(row):
        column_name = headers[col_index] if headers[col_index] else f"Column{col_index}"
        item[column_name] = str(value) if value is not None else ""
    table.put_item(Item=item)

# Send notification to management
message = f"New upload from {company_id} processed with {len(data_rows)} records."
sns.publish(
    TopicArn=SNS_TOPIC_ARN,
    Subject="New Upload Notification",
    Message=message
)
```

```
    return {
        'statusCode': 200,
        'body': json.dumps('Excel processed successfully!')
    }
```

Code used for IAM Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject"],
      "Resource": ["arn:aws:s3:::xyz-logistics-uploads-yourname/companyA/*"]
    }
  ]
}
```

Code for front end

Index code

```
<!DOCTYPE html>
<html>
<head>
  <title>Upload Excel File</title>
</head>
<body>
  <h2>Upload Excel File for Company A</h2>
  <input type="file" id="fileUpload" />
  <button onclick="uploadFile()">Upload</button>
  <p id="status"></p>

  <script src="https://sdk.amazonaws.com/js/aws-sdk-2.1180.0.min.js"></script>
  <script src="upload.js"></script>
```

```
</body>
</html>
```

Upload.js

```
AWS.config.update({
  accessKeyId: 'YOUR_ACCESS_KEY',
  secretAccessKey: 'YOUR_SECRET_KEY',
  region: 'us-east-1'
});

const s3 = new AWS.S3();

function uploadFile() {
  const file = document.getElementById("fileUpload").files[0];
  const companyPrefix = 'companyA/';

  if (!file || !file.name.endsWith('.xlsx')) {
    document.getElementById("status").innerText = "Please select a valid .xlsx file";
    return;
  }

  const params = {
    Bucket: 'xyz-logistics-uploads-chamith',
    Key: companyPrefix + file.name,
    Body: file,
    ContentType: file.type
  };

  s3.upload(params, function(err, data) {
    if (err) {
```

```
    document.getElementById("status").innerText = "Upload failed: " + err.message;
  } else {
    document.getElementById("status").innerText = "Upload successful!";
  }
});
}
```

Screenshots

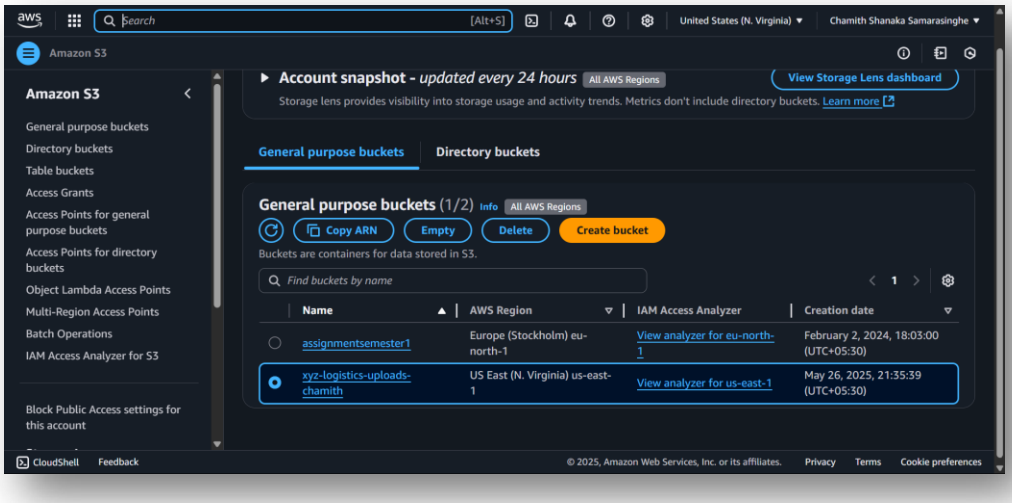

Figure 2: S3 Bucket created

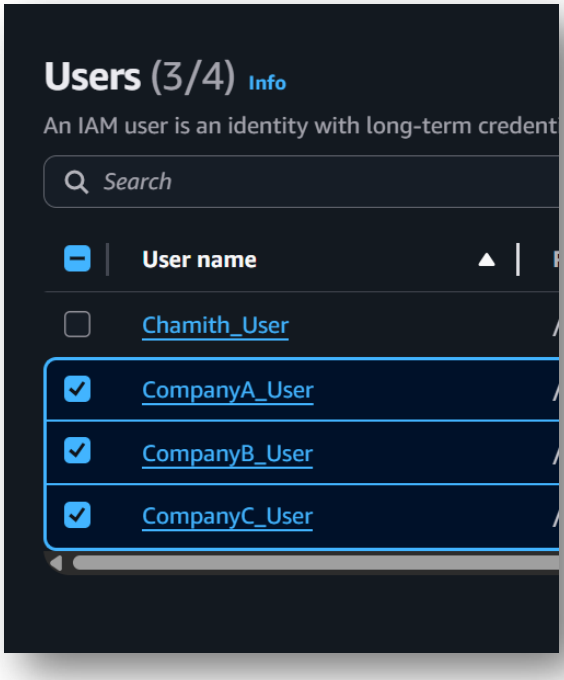

Figure 3: IAM Users created

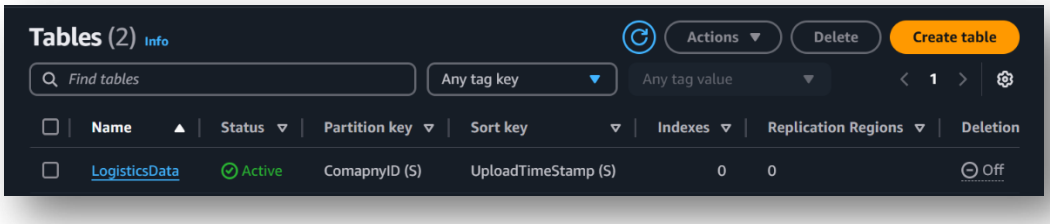

Figure 4: DynamoDB table created

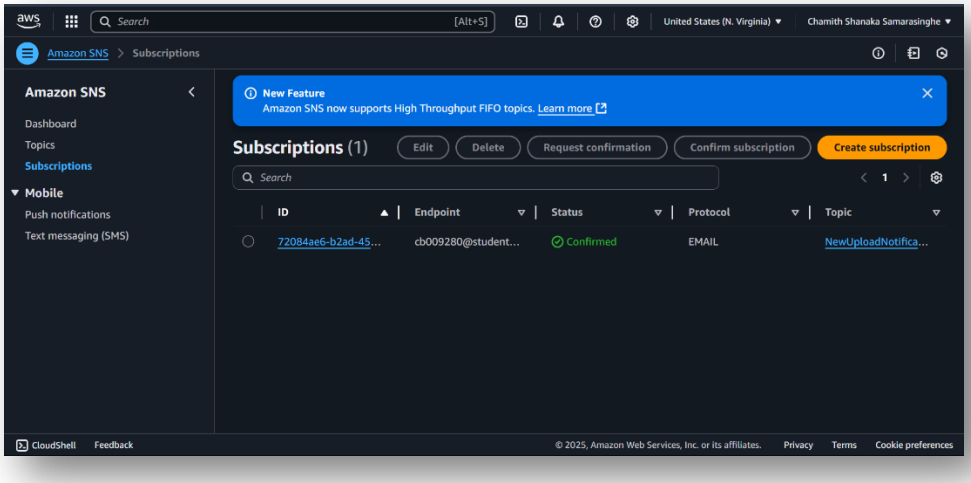

Figure 5: SNS Subscription created

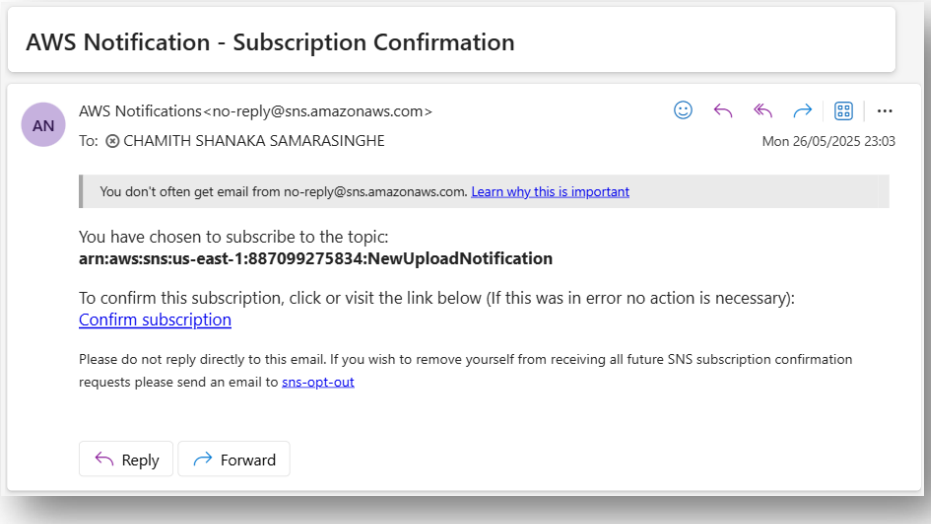

Figure 6: SNS Subscription E-Mail Confirmation

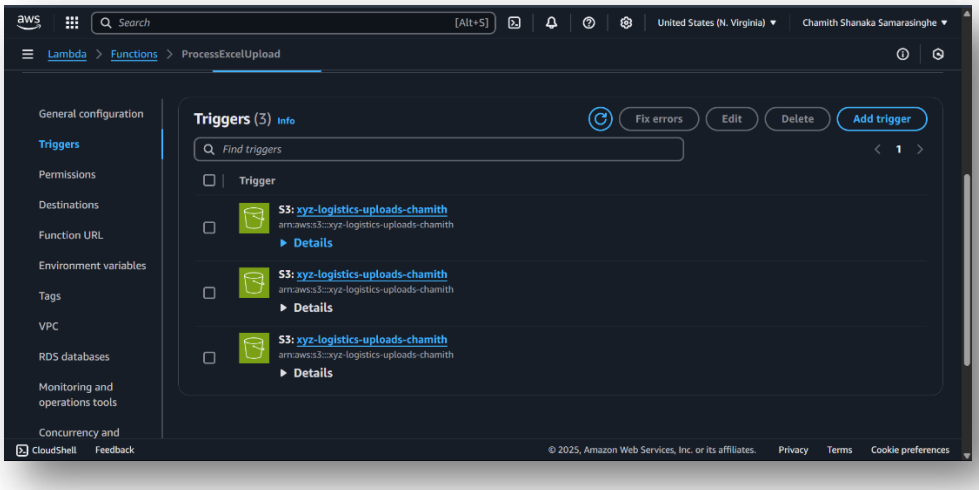

Figure 7: Triggers for companyA, companyB, company

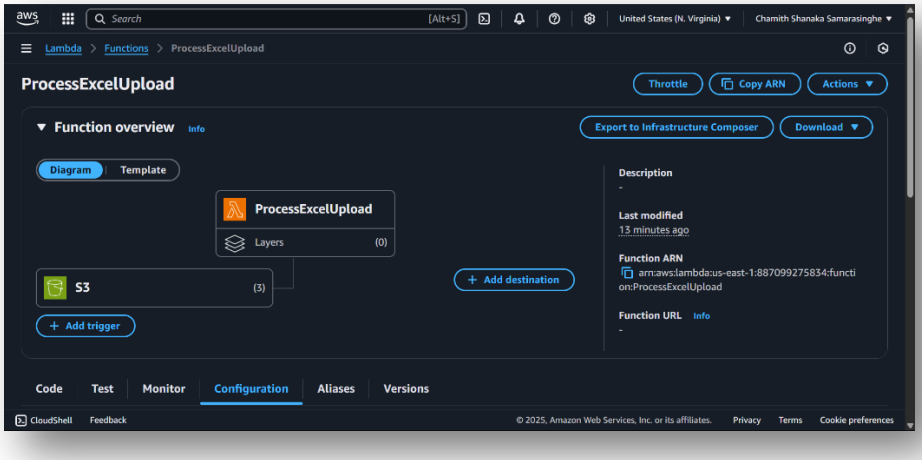

Figure 8: Lambda Function Created

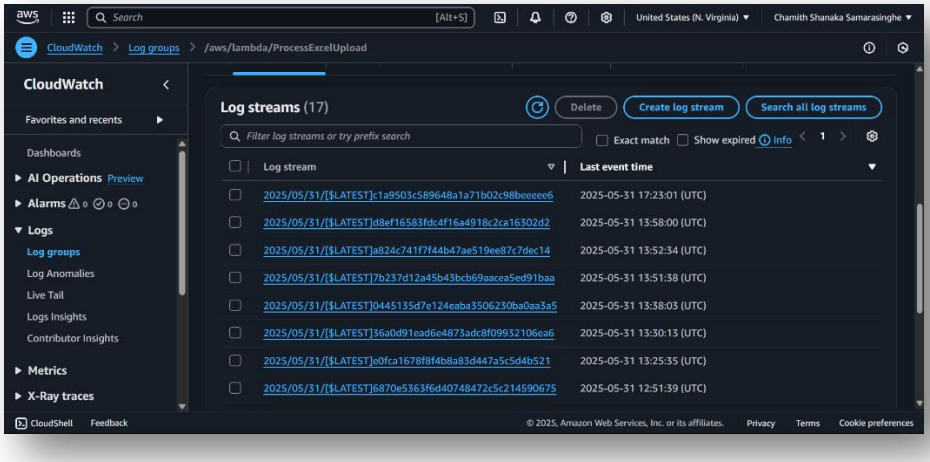

Figure 9: Log Streams after each test and uploads of excel file

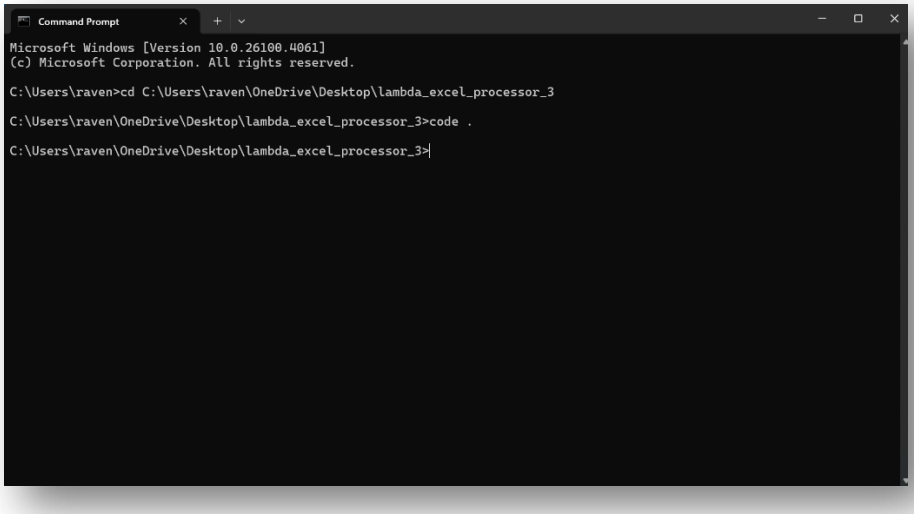

Figure 10: Created a code file in the folder

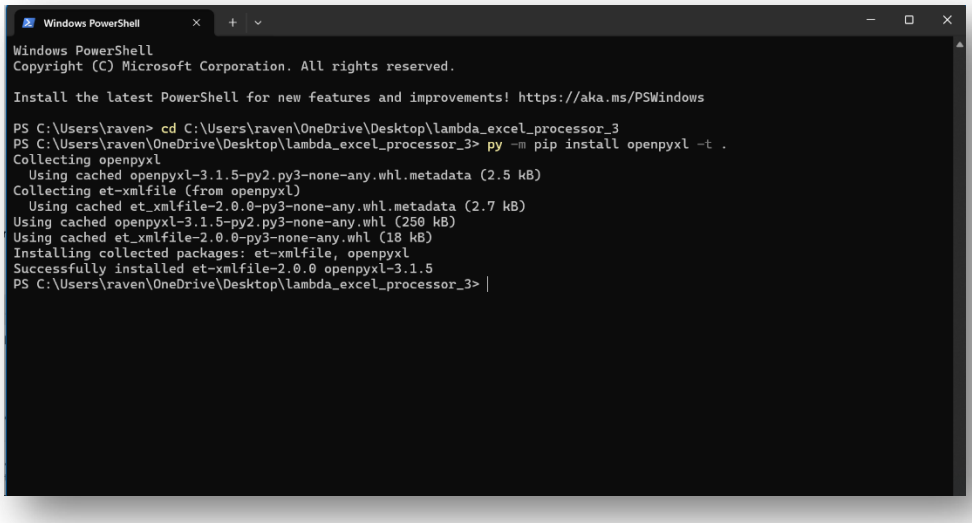

Figure 11: Installed the necessary python dependencies

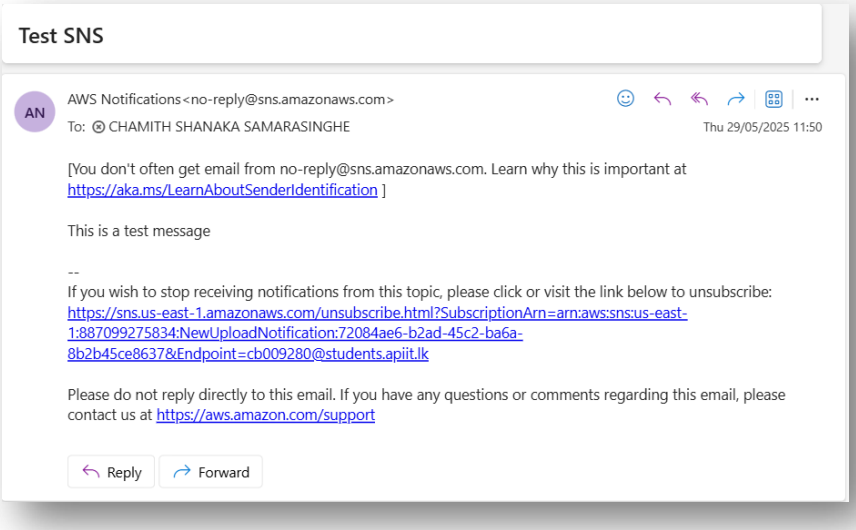

Figure 12: Test SNS E-Mail

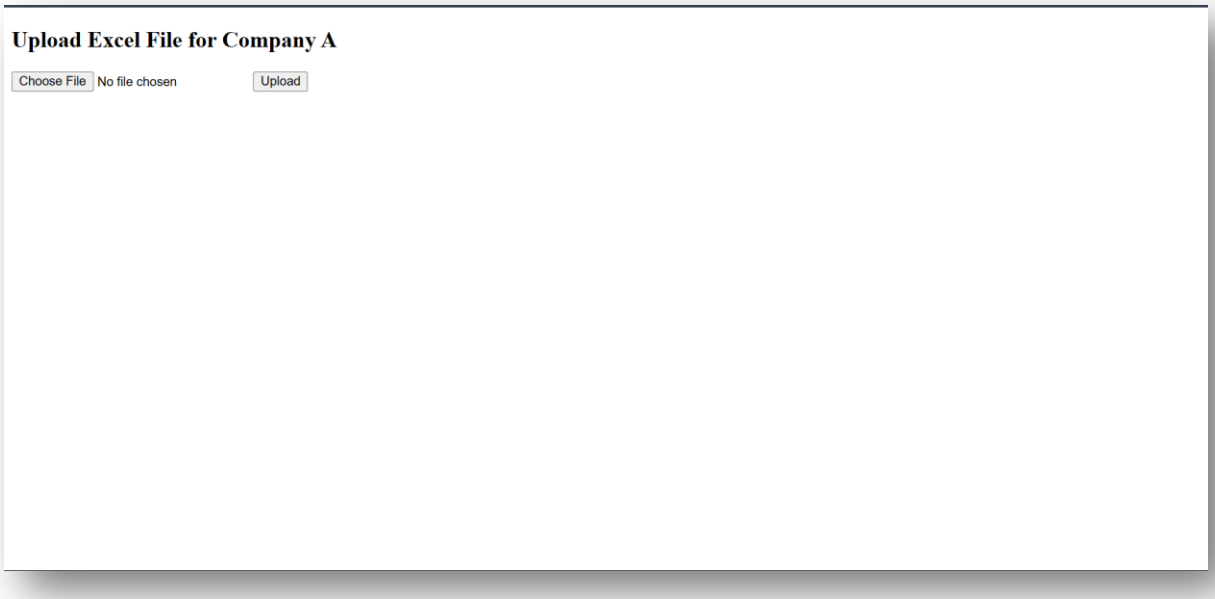

Figure 13: Frontend

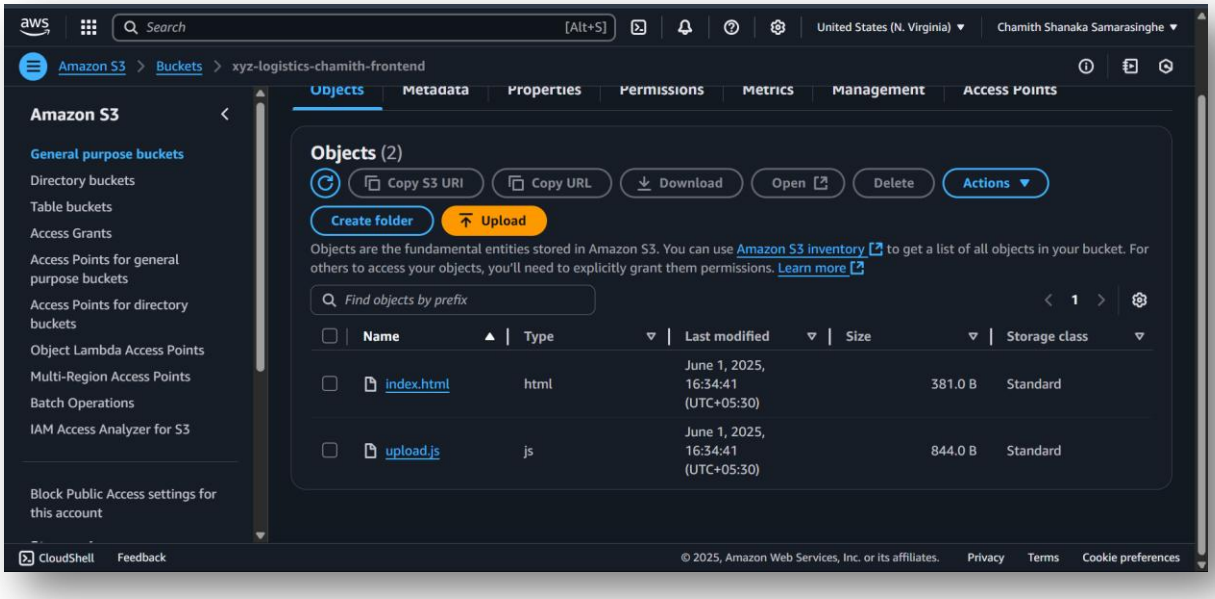

Figure 14: New S3 Bucket for the frontend